

Seminar

RAII. Rule of 0, 3, and 5.

1. Resource Acquisition Is Initialization.

- From cppreference.com:
*Resource Acquisition Is Initialization or **RAII**, is a C++ programming technique which binds the life cycle of a resource that must be acquired before use (allocated heap memory, thread of execution, open socket, open file, locked mutex, disk space, database connection - anything that exists in limited supply) to the lifetime of an object.*

RAII can be summarized as follows:

- **encapsulate** each resource into a **class**, where
 - the constructor acquires the resource and establishes all class invariants or throws an **exception** if that cannot be done,
 - the destructor releases the resource and **never throws** exceptions;
- **always** use the resource via an instance of a **RAII-class** that either
 - has automatic storage duration or temporary lifetime itself, or
 - has lifetime that is bound by the lifetime of an automatic or temporary object

2. Const methods.

- Every method that does **NOT** change the object should be declared as **const**.
- Example in the header file:
`void print() const;`
- Example in the source file:

```
void ClassName::print() const {  
    cout << <member>;  
}
```

3. Initializer list.

- Used for initializing const members or any member that needs to be initialized before the body of the constructor.
- Separated by commas, used like a constructor for the members.
- Other constructors can be called using the initializer list.
- Example:
`Person::Person() : age(0), name(nullptr)
{ ... }`
- Example2:
`Person::Person(int age, const char* name) : Person()
{ ... }`

4. Copy constructor.

- How our object will be copied when it's passed by value.
- Usually implemented via a copy method that can be used in the operator=.
- In code:

```
ClassName::ClassName(const ClassName& other)
{
    copyData(other);
}
```

5. Operator=.

- Overloading the '=' operator. *More on overloading operators in the next lesson.*
- In code:

```
ClassName& ClassName::operator=(const ClassName& other)
{
    if (this != &other)    // Important check!
    {
        clearData();
        copyData(other);
    }
    return *this;
}
```

```
ClassName c1("Sample text");
ClassName c2;
c2 = c1;    // operator= is called
```

```
ClassName c1("Sample text");
ClassName c2 = c1;    // Copy constructor is called, because
                     // ClassName c2 = c1; ⇔ ClassName c2(c1);
```

6. Destructor.

- Called once the object is deleted via `delete` for dynamically allocated objects **or** once the program exits the code block where the object was instantiated.
- Mainly used for deleting dynamically allocated data members.
- Usually implemented via a clear method that is also used in the operator=.
- In code:

```
ClassName::~~ClassName()
{
    clearData();
}
```

7. The big four (The rule of 3).

- A class is in a **canonical form** if it implements **the big four**:
 - default constructor, copy constructor, operator= and destructor.
- As a rule of thumb we implement the big four when the class has at least one data member that **cannot** be copied trivially, such as dynamically allocated memory. So the object can create a **deep copy** of itself rather than **shallow**.
- If our class does **not** need a special deep copy implementation, then we do **not write any** of the three - copy constructor, operator= and destructor.

8. Exceptions.

- Should be used only in **exceptional** cases.
- Constructors **can throw** exceptions to stop the creation of an object.
- Destructors **should not** throw exceptions.
- Can be caught with a try - catch block.
- In code:

```
try
{
    // some code that could throw an exception
}
catch (std::bad_alloc& ex)
{
    std::cout << ex.what() << '\n';
    std::cout << "Couldn't do something, continuing...";
}
catch (...)
{
    std::cout << "Caught an unexpected exception, rethrowing...";
    throw; // Rethrowing the same exception
}
```